**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ**
**(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

Институт № 8 «Компьютерные науки»

**ЛАБОРАТОРНАЯ РАБОТА № 1**

по курсу «Программирование графических процессоров»

# «Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами»

Выполнил: Д. А. Кулешов
Группа: М8О-408Б-23
Преподаватель: А. Ю. Морозов

Москва 2026

## 1 Условие

Цель работы: освоить основные средства CUDA и научиться выполнять простые операции над векторами на графическом процессоре.

Условие задания представлено на рисунке 1.

**Вариант 5. Поэлементное нахождение максимума векторов.**

**Входные данные.** На первой строке задано число n -- размер векторов. В следующих 2-х строках, записано по n вещественных чисел -- элементы векторов.

**Выходные данные.** Необходимо вывести n чисел -- результат поэлементного нахождения максимума исходных векторов.

**Пример:**

| Входной файл | Выходной файл |
|---|---|
| 3<br>1 5 3<br>4 2 6 | 4.0000000000e+00 5.0000000000e+00 6.0000000000e+00 |

Рисунок 1 — Условие задания

## 2 Программное и аппаратное обеспечение

### 2.1 Аппаратное обеспечение

Характеристики использованного аппаратного обеспечения приведены в таблице 1.

Таблица 1 — Характеристики аппаратного обеспечения

| Характеристика | Значение |
|---|---|
| Процессор | AMD Ryzen 5 5600, 6 ядер, 12 потоков |
| Кэш процессора | L1d — 192 KiB; L1i — 192 KiB; L2 — 3 MiB; L3 — 32 MiB |
| Оперативная память | 32 GiB; в WSL доступно 15 GiB; swap 4 GiB |
| Накопитель | INTEL SSDPEKNU512GZ, 512 GB |
| Графический процессор | NVIDIA GeForce RTX 5070 |
| Compute capability | 12.0 |
| Графическая память | 12 199 MiB |
| Разделяемая память на блок | 49 152 байт (48 KiB) |
| Константная память | 65 536 байт (64 KiB) |
| Регистров на блок | 65 536 |

| Характеристика | Значение |
|---|---|
| Максимальное число нитей в блоке | 1024 |
| Максимальный размер сетки | 2 147 483 647 × 65 535 × 65 535 блоков |
| Число мультипроцессоров | 48 |

Характеристики GPU были получены с помощью `nvidia-smi` и функции `cudaGetDeviceProperties`.

### 2.2 Программное обеспечение

Использованное программное обеспечение приведено в таблице 2.

Таблица 2 — Использованное программное обеспечение

| Компонент | Значение |
|---|---|
| Операционная система | Ubuntu 26.04.1 LTS в WSL |
| Архитектура | x86-64 |
| CUDA Toolkit | 13.4 |
| Компилятор CUDA | nvcc 13.4.92 |
| Драйвер NVIDIA | 616.92 |
| Среда разработки | Visual Studio Code |

## 3 Метод решения

На выход необходимо получить вектор. Покоординатно сравниваем элементы исходных векторов `arr1` и `arr2`. Покоординатные максимумы записываем в соответствующие координаты в `result_arr`.

Значение каждого элемента результирующего вектора не зависит от других элементов. Поэтому разные пары чисел можно сравнивать одновременно. На CPU они обрабатываются по очереди в обычном цикле, на GPU работа распределяется между множеством нитей.

Каждая нить сначала вычисляет свой индекс:

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

Затем вычисляется число всех запущенных нитей:

```
int offset = blockDim.x * gridDim.x;
```

После обработки одного элемента выполняется `idx += offset`. Поэтому нить переходит к следующему элементу, который относится к ней.

Последовательный алгоритм имеет сложность $O(n)$. GPU выполняет ту же общую работу, но распределяет её между большим числом нитей.

## 4 Описание программы

Работа состоит из трёх исходных файлов:

- `lab1.cu` — сама лабораторная;
- `benchmark.cu` — проверка и сравнение GPU- и CPU-решения;
- `properties.cu` — вывод характеристик GPU.

Векторы хранятся в массивах типа `float`. Переменная `n` типа `int` хранит количество элементов. Указатели с приставкой `dev_` относятся к памяти GPU, остальные указатели — к оперативной памяти компьютера.

### 4.1 CUDA-ядро

В программе реализовано одно ядро `kernel`:

```
__global__ void kernel(double *arr1, double *arr2, double
*result_arr, int n) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int offset = blockDim.x * gridDim.x;

    while (idx < n) {
        result_arr[idx] = arr1[idx] > arr2[idx] ? arr1[idx] :
arr2[idx];
        idx += offset;
    }
}
```

### 4.2 CPU-версия

В CPU-версии используется обычный цикл с условием:

```
void cpu_max(double *arr1, double *arr2, double *result, int n)
{
    for (int idx = 0; idx < n; ++idx) {
        result[idx] = arr1[idx] > arr2[idx] ? arr1[idx] :
arr2[idx];
```

```
    }
}
```

## 5 Результаты

### 5.1 Проведение замеров

Время работы ядра измерялось с помощью CUDA Events. Перед каждым замером ядро один раз запускалось для прогрева. Для каждого массива выполнялось 100 повторов. В таблице 3 указано среднее время одного запуска. Копирование данных между CPU и GPU в это время не входит.

Время CPU-версии измерялось с помощью `chrono`. Вывод данных в консоль не измерялся.

Все программы были собраны с уровнем оптимизации `-O2`.

### 5.2 Время работы CUDA-ядра

Таблица 3 — Среднее время работы CUDA-ядра

| Конфигурация | $n$ = 1 000, мс | $n$ = 1 000 000, мс | $n$ = 100 000 000, мс |
|---|---|---|---|
| `<<<1, 32>>>` | 0,013005 | 4,650862 | 1018,690918 |
| `<<<32, 32>>>` | 0,008221 | 0,147555 | 33,046387 |
| `<<<64, 128>>>` | 0,010842 | 0,024081 | 4,978114 |
| `<<<128, 256>>>` | 0,008022 | 0,011783 | 2,202348 |
| `<<<256, 512>>>` | 0,007284 | 0,011443 | 2,254116 |
| `<<<1024, 1024>>>` | 0,009361 | 0,014301 | 2,110281 |

### 5.3 Сравнение с CPU

Ускорение вычислялось по формуле

$$S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}} \tag{1}$$

где $T_{\text{CPU}}$ — время работы CPU-версии; $T_{\text{GPU}}$ — время работы CUDA-ядра.

Результаты сравнения приведены в таблице 4.

Таблица 4 — Сравнение времени работы CPU и GPU

| Размер $n$ | CPU, мс | GPU, мс | Лучшая конфигурация | Ускорение |
|---|---|---|---|---|
| 1 000 | 0,000122 | 0,007284 | `<<<256, 512>>>` | 0,017 |
| 1 000 000 | 0,154780 | 0,011443 | `<<<256, 512>>>` | 13,53 |
| 100 000 000 | 40,936608 | 2,110281 | `<<<1024, 1024>>>` | 19,40 |

На маленьком массиве CPU работает быстрее. Само вычисление занимает очень мало времени, поэтому расходы на запуск CUDA-ядра оказываются больше полезной работы. На среднем массиве GPU быстрее в 13,53 раза, а на большом — в 19,40 раза. Сравнивается время вычисления, время копирования массивов между CPU и GPU не учитывается.

Конфигурация `<<<1, 32>>>` плохо подходит для больших массивов, потому что в ней запускается всего 32 нити.

Для маленького массива слишком большая конфигурация тоже невыгодна, запуск лишних нитей не ускоряет вычисления.

## 6 Выводы

В работе были созданы CUDA- и CPU-версии программы для решения задачи. Задача хорошо подходит для параллельной обработки, так как все элементы результата вычисляются независимо друг от друга.

Для небольшого массива использование GPU не дало преимущества. Для массива из 1 000 000 элементов было получено ускорение в 13,53 раза, а для массива из 100 000 000 элементов — в 19,40 раза. GPU выгоднее использовать для достаточно больших объёмов данных.

Исходный код работы: https://github.com/dmitriikuleshov/pgp-lab1